Precious Fikile Mkhize

0210210785080

DEMONSTRATE KNOWLEDGE OF THE PRINCIPLES OF ELECTRONIC LOGIC FOR COMPUTING

US 14922

Electronic logic for computing relies on the binary number system, Boolean algebra, and physical switching devices to process data. [1, 2]

Binary System and Boolean Logic

- **Binary Base-2 System:** Computers represent all data and instructions using two discrete voltage states: **1** (True, High, or presence of voltage) and **0** (False, Low, or absence of voltage).
- **Transistors:** Microscopic semiconductor switches inside computer chips turn current *on* (1) or *off* (0), serving as the physical foundation for computation.
- **Boolean Algebra:** A mathematical system that formulates logical decisions using operators like AND, OR, and NOT. [1, 3, 4, 5, 6, 7]

Core Logic Gates

Logic gates are physical circuits built from transistors that accept binary inputs and yield a single binary output. [6]

- **NOT Gate (Inverter):** Takes 1 input and flips it (Input 0 → Output 1; Input 1 → Output 0).
- **AND Gate:** Requires 2 or more inputs; outputs **1** only if *all* inputs are 1.
- **OR Gate:** Requires 2 or more inputs; outputs **1** if *at least one* input is 1.
- **NAND Gate:** An AND gate followed by a NOT gate; outputs **0** only when all inputs are 1.
- **NOR Gate:** An OR gate followed by a NOT gate; outputs **1** only when all inputs are 0.
- **XOR (Exclusive OR) Gate:** Outputs **1** only when the inputs are *different* from each other (one is 1, the other is 0). [3, 8]

Truth Tables

A truth table maps every possible combination of binary inputs to its resulting output. For example, a 2-input AND gate truth table: [4, 7]

Input A	**Input B**	**Output (A AND B)**
0	0	0
0	1	0

1	0	0
1	1	1

Combinational vs. Sequential Circuits

- **Combinational Circuits:** Outputs depend *only* on the current input values at that exact moment (e.g., adders, multiplexers). They have no memory.
- **Sequential Circuits:** Outputs depend on both current inputs *and* past states, allowing the circuit to store information (e.g., flip-flops, registers, RAM memory chips). [2, 9]

Here are two options to choose from. We can either simplify a complex logic circuit using a **Karnaugh Map (K-Map)**, or work through a **truth table exercise** step-by-step.

Option 1: Karnaugh Map (K-Map) Simplification

A Karnaugh map is a visual grid used to simplify Boolean expressions without doing tedious algebra. Let's simplify the following expression for a 3-input circuit (A, B, C):
$$F = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + AB\bar{C} + ABC$$

Step 1: Map the 1s into the grid

We place a **1** in the cells corresponding to the terms in our expression. Notice that the columns are ordered using **Gray code** (00, 01, 11, 10) so only one bit changes at a time.

A \ BC	**00 ($\bar{B}\bar{C}$)**	**01 ($\bar{B}C$)**	**11 (BC)**	**10 ($B\bar{C}$)**
0 ($\bar{A}$)	**1**	0	0	**1**
1 (A)	0	0	**1**	**1**

Step 2: Group the 1s

We group adjacent 1s in sizes of powers of two (2, 4, 8). Groups can wrap around the edges like a cylinder.

1. **Horizontal pair on the top row:** $\bar{A}\bar{B}\bar{C}$ and $\bar{A}B\bar{C}$ wrap around to make a group of 2. In this group, B changes (0 to 1), so it drops out. A is 0 and C is 0. **Result = $\bar{A}\bar{C}$**
2. **Vertical pair on the right:** $\bar{A}B\bar{C}$ and $AB\bar{C}$ make a group of 2. A changes, so it drops out. B is 1 and C is 0. **Result = $B\bar{C}$**
3. **Horizontal pair on the bottom row:** $AB\bar{C}$ and ABC make a group of 2. C changes, so it drops out. A is 1 and B is 1. **Result = AB**

Step 3: Write the final simplified expression

$$F = \bar{A}\bar{C} + B\bar{C} + AB$$

Option 2: Step-by-Step Truth Table Exercise

Let's break down how a combination of gates behaves. We will find the output Y for the logic circuit expression:
$$Y = (A \text{ AND } B) \text{ OR } (\text{NOT } B)$$
To solve this, we break it into smaller sub-expressions columns:

Input A	**Input B**	**Column 1: $(A \text{ AND } B)$**	**Column 2: $(\text{NOT } B)$**	**Final Output Y: (Col 1 OR Col 2)**
0	0	0	1	**1**
0	1	0	0	**0**
1	0	0	1	**1**
1	1	1	0	**1**